

Deployment Portfolio Task 4

Student Name: Alex Phan

Student ID: 105334704

Unit Code & Title: SWE40006 Software Deployment and Evolution

Semester & Submission Date: Semester 2, 15/09/2026

Declared Target Level: Distinction (4.3)

Execution Evidence:

Pass Level Task:

WSL installation

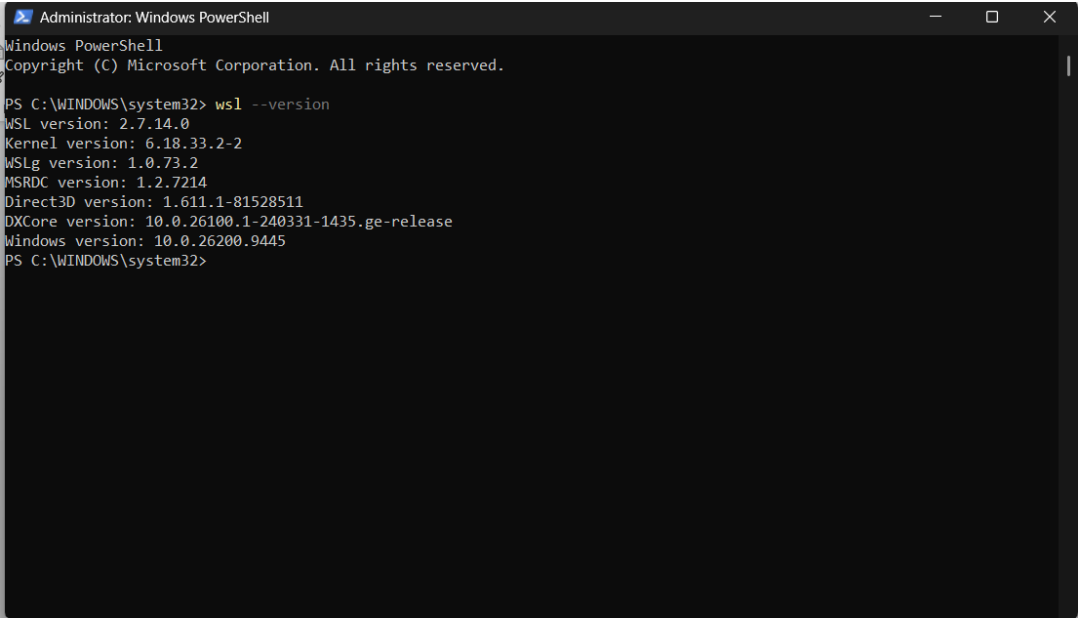


Figure 1, wsl installation

The figure shows that wsl is installed and the version is newer than 2.1.5.

Docker desktop installation

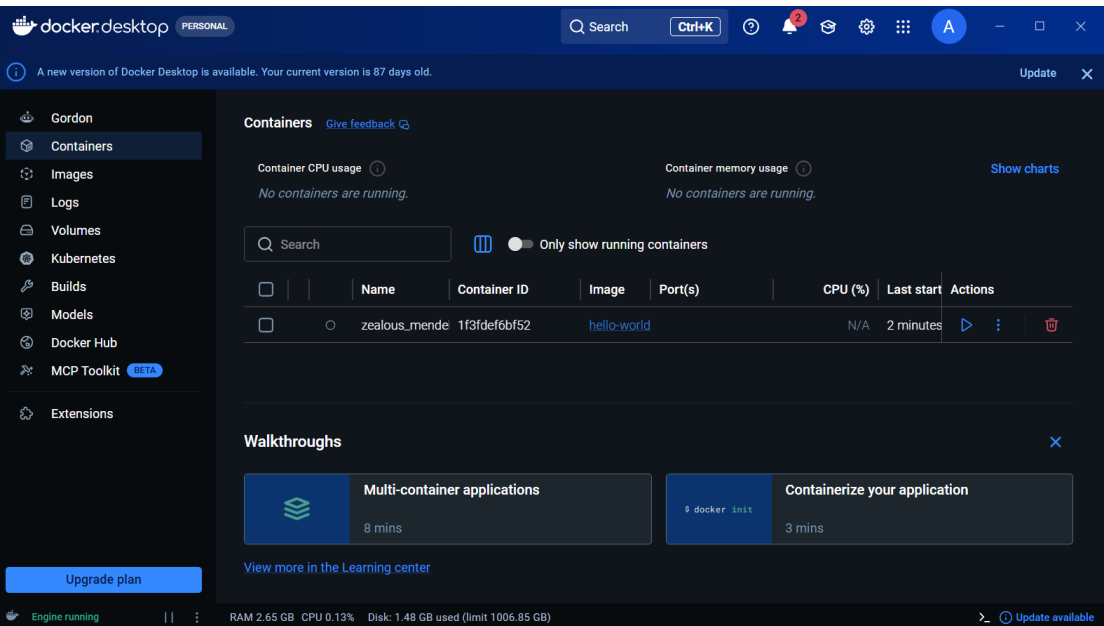
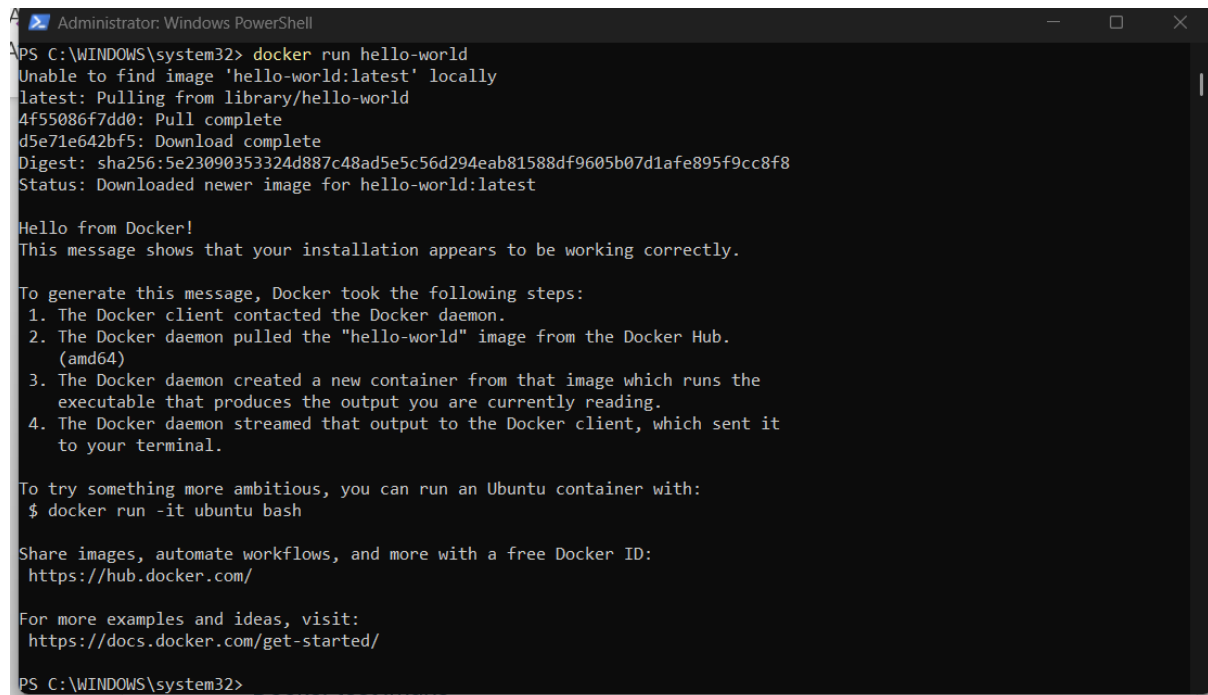


Figure 2, Docker desktop

Figure 2 shows the docker desktop installation being completed and is running the test container hello-world downloaded from docker desktop.

Docker test image



```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

PS C:\WINDOWS\system32>
```

Figure 3, test image

Figure 3 shows that I have pulled an official hello-world container image from docker hub, additionally figure 2 also shows that the container has been registered within docker desktop.

Credit Level Task:

Sample Flask Application

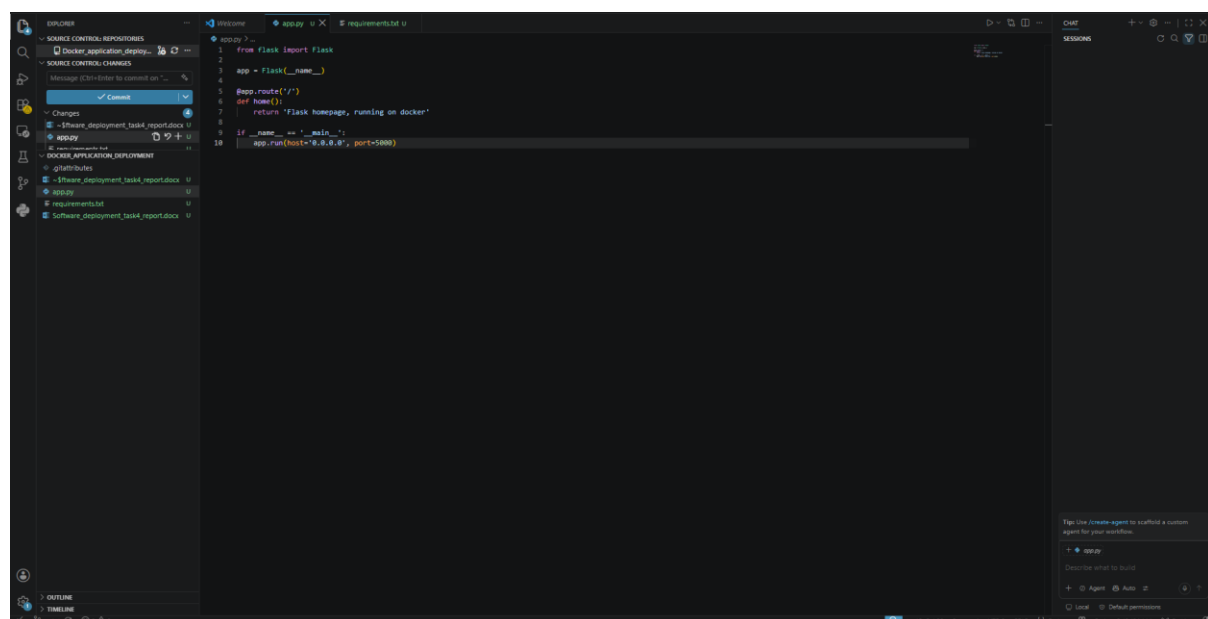


Figure 4, Flask App

First, I have created the app.py file and defined the standard flask routing code which prints a welcome statement on a webpage running on flask, the flask allows connection on any interface.

Requirements file.

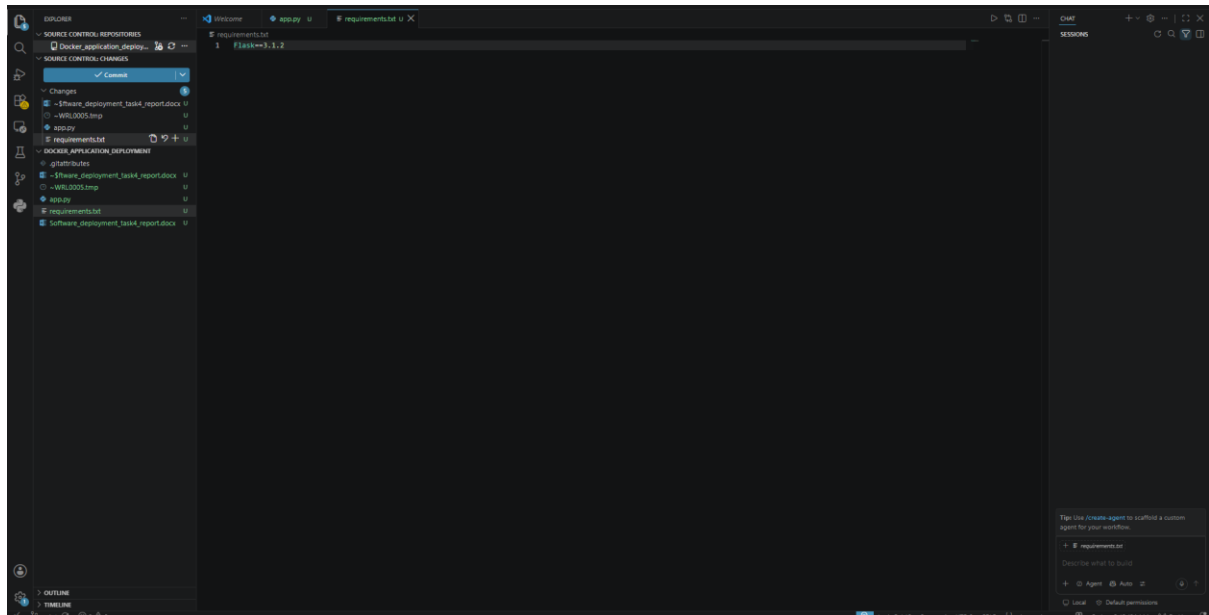


Figure 5, Requirements file.

Created the requirements.txt file which contains the flask version needed to run the web app. This is for docker when the container spun up it will check the requirements file and makes sure to install all dependencies needed to run the application.

Docker File

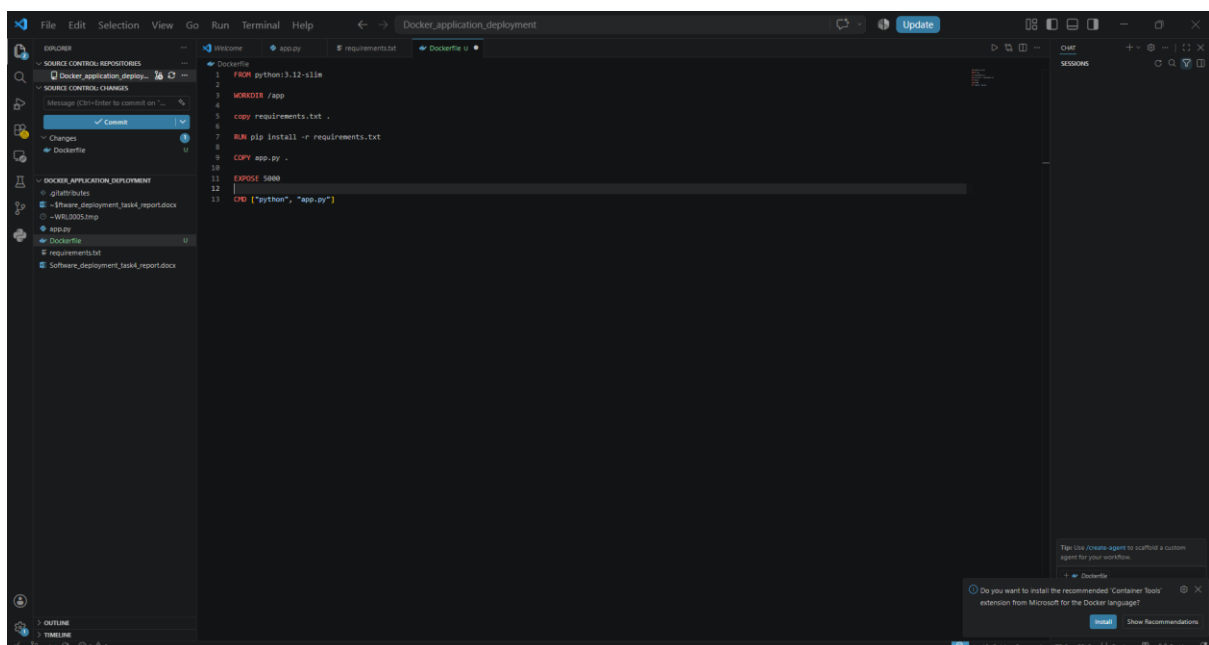


Figure 6, Docker file

Figure 6 shows the specifications for the Docker file defined in the working directory, where it will use a standard python image, create a working directory, download requirements, let docker know which port to listen for and run the python flask application after spinning up the container.

Docker Image of flask application

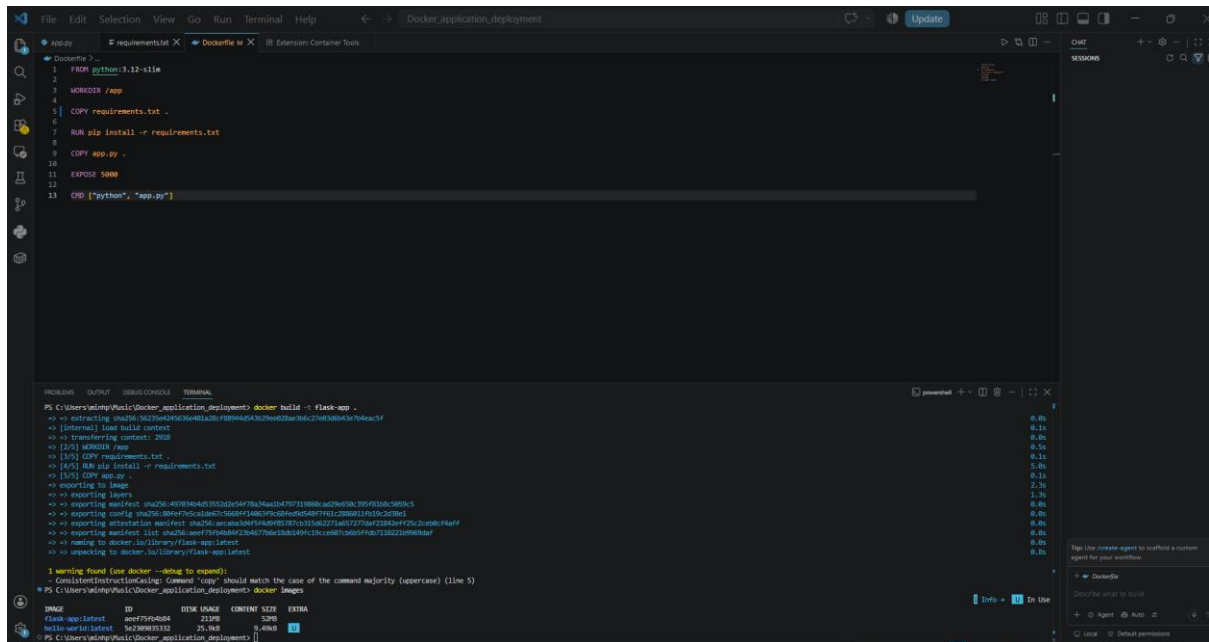


Figure 7, Docker Flask Image

Figure 7 portrays the docker flask image stored in the docker desktop virtualized storage after running the build command to build the template of the flask application, so that when docker spins up containers and use this template to correctly build and configure the application.

Docker Container creation

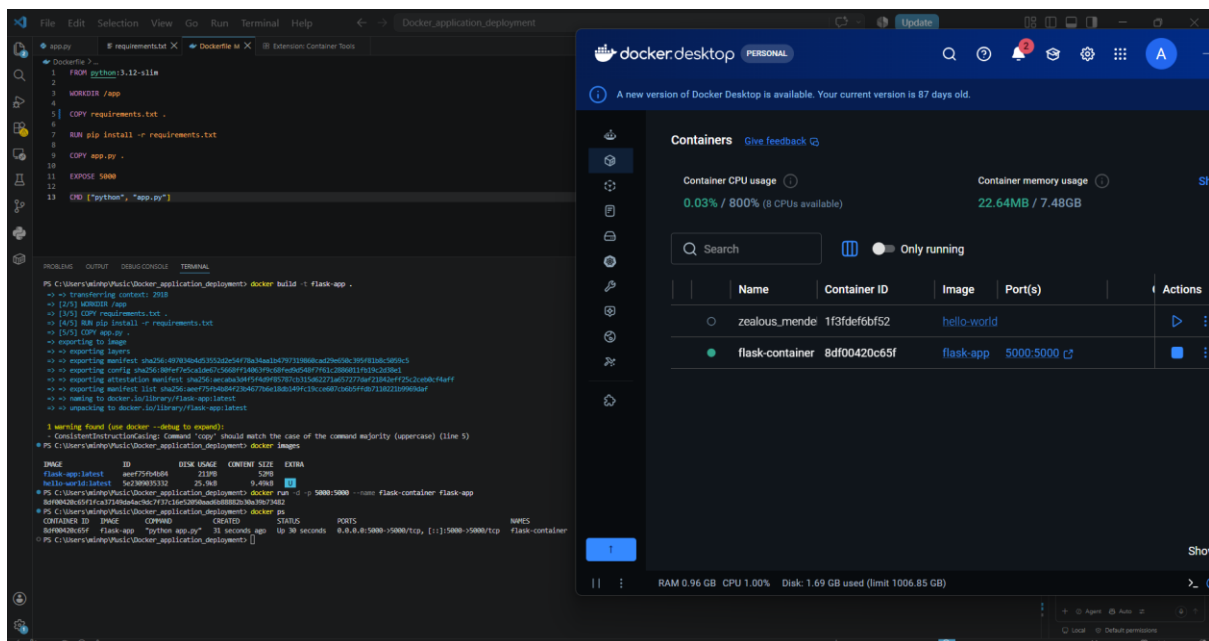


Figure 8, Docker Container

The docker container was then created with the application image illustrated in figure 7, I have also mapped the ports so that to access the container I would access through the same port the flask web server is running on to simplify the accessibility.

Flask App on docker

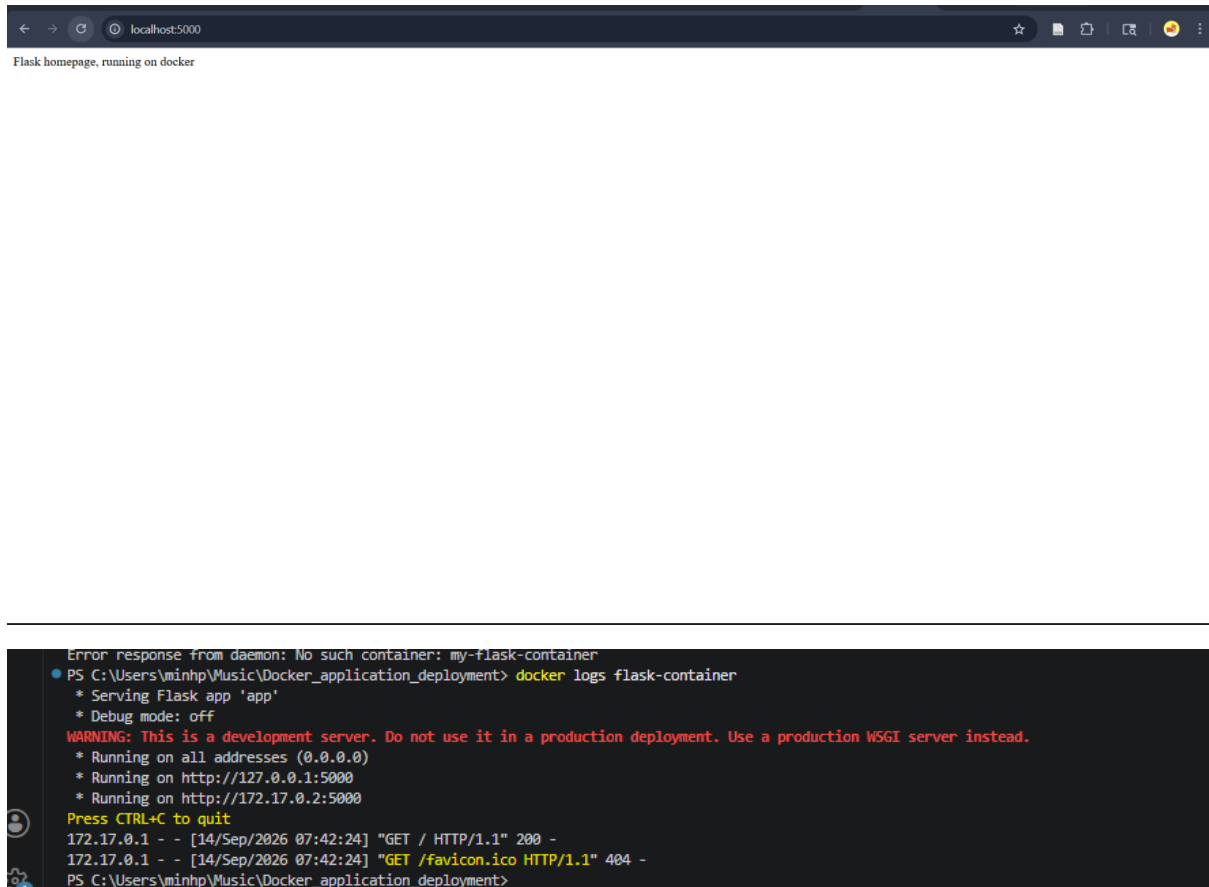


Figure 9, Containerized Flask app

Figure 9 shows that the flask app is accessible but also the docker logs highlight that the web is app is successfully running on a docker container.

Docker repo

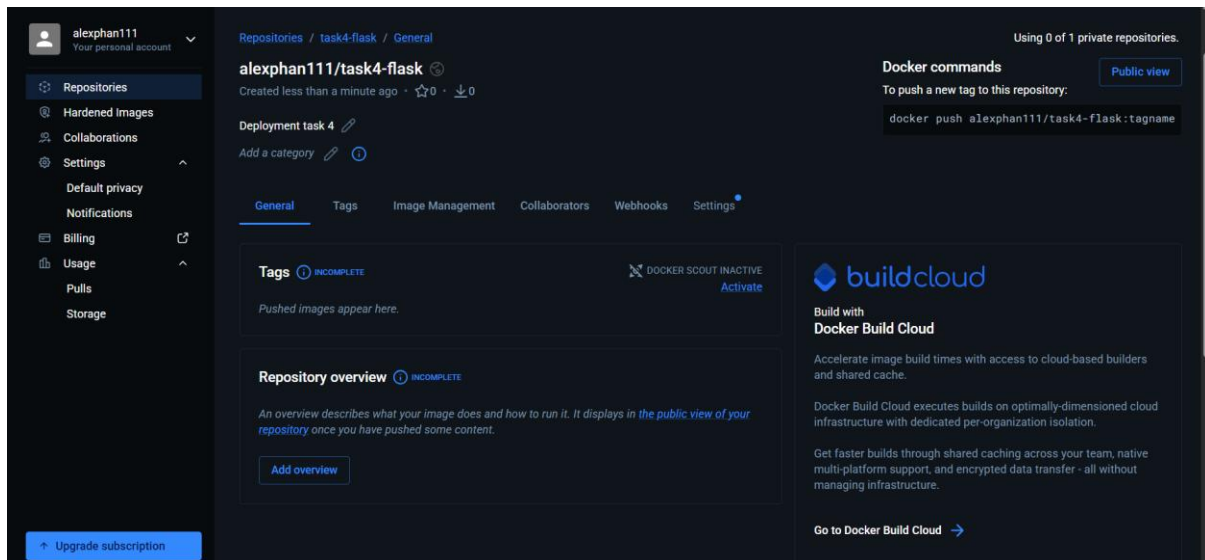


Figure 10, Repo

Create a repo on docker hub to be able to push the image to.

Push image

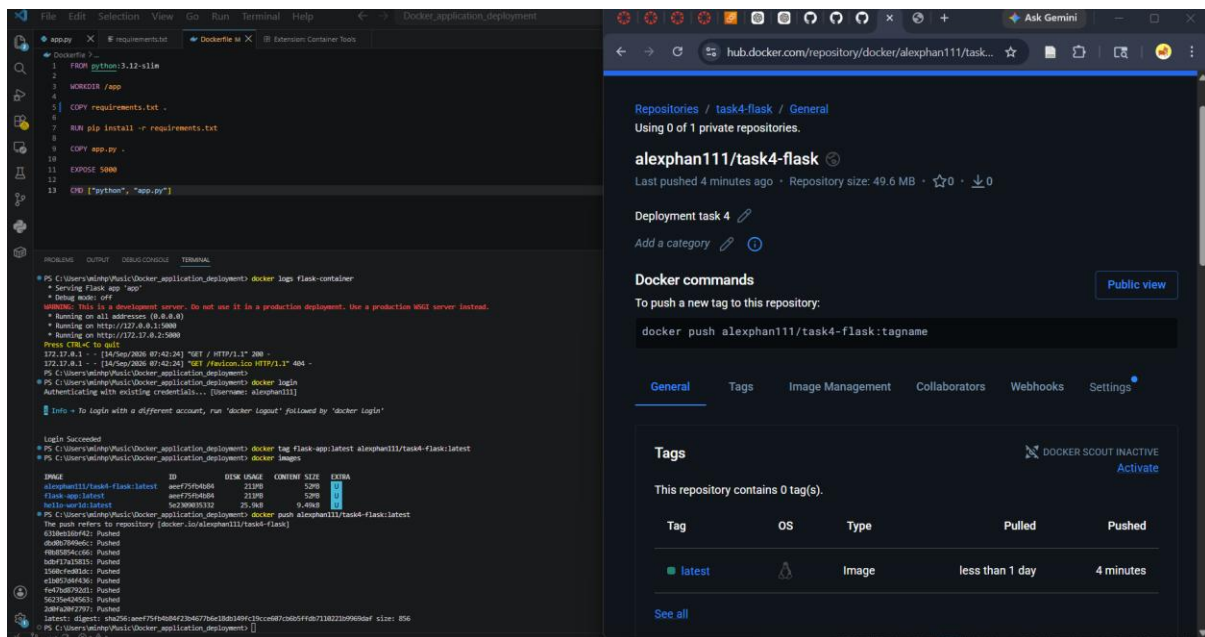


Figure 11, Pushed image.

Figure 10 shows the repo now storing the flask image, this can now be used for a secondary docker host.

Login on docker on new environment

```
Problems Output Debug Console Terminal Ports
PS C:\Users\Alex\Music\ComputingTechnology\ProjectBR45> docker login
Authenticating with existing credentials... [Username: endemzzzz99]

Info -> To Login with a different account, run 'docker Logout' followed by 'docker Login'

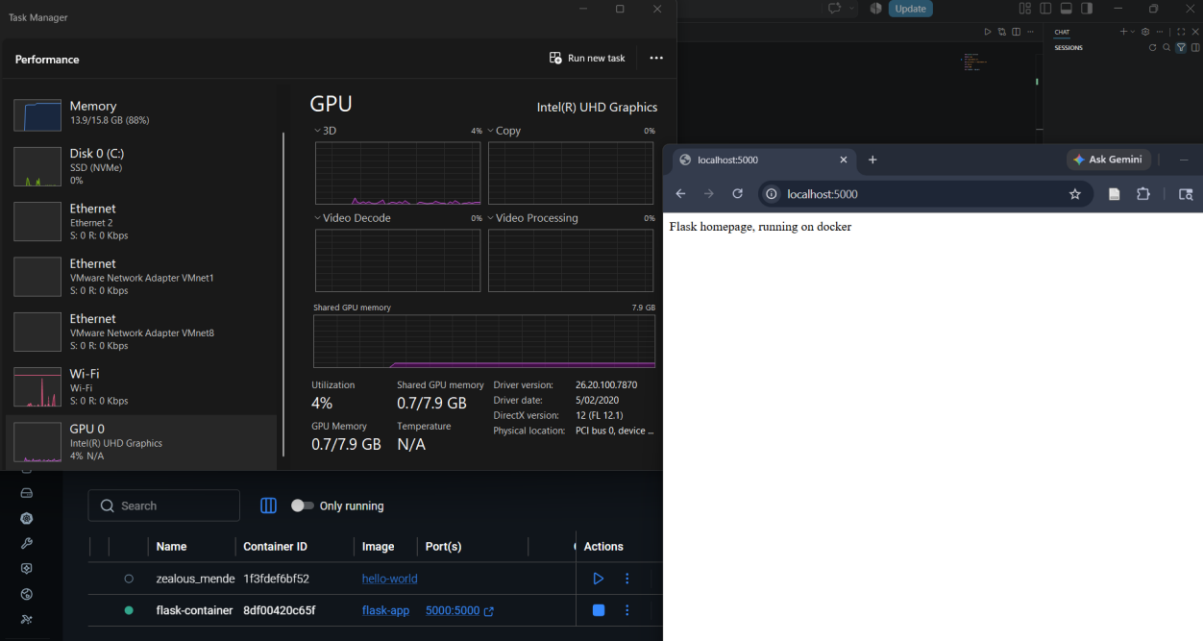
Login Succeeded
PS C:\Users\Alex\Music\ComputingTechnology\ProjectBR45> docker pull alexphan111/task4-Flask:latest
latest: Pulling from alexphan111/task4-Flask
2a0f920f2797: Pulling fs layer
2a0f920f2797: Pull complete
f0b85854cc66: Pull complete
dcd0b7849e6c: Pull complete
1560cfe90dcd: Pull complete
b0bf17a15815: Pull complete
e1b05704f436: Pull complete
6310eb16bf42: Pull complete
fe47bd8792d1: Download complete
Digest: sha256:aeef79fb4b4f23b4677b6e18db149fc19ccc607cb5b5ffdb7110221b0969d4f
Status: Downloaded newer image for alexphan111/task4-Flask:latest
docker.io/alexphan111/task4-Flask:latest

What's next:
View a summary of image vulnerabilities and recommendations -> docker scout quickview alexphan111/task4-Flask:latest
PS C:\Users\Alex\Music\ComputingTechnology\ProjectBR45> docker run -d -p 5000:5000 --name task4-container alexphan111/task4-Flask:latest
3232b986b2193e8d7e7de46ee240fd0dc7d2903814b3575a61149235430d9ae
PS C:\Users\Alex\Music\ComputingTechnology\ProjectBR45> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
3232b986b219   alexphan111/task4-Flask:latest      "python app.py"         25 seconds ago Up 24 seconds 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  task4-container
PS C:\Users\Alex\Music\ComputingTechnology\ProjectBR45>
```

Figure 12, New environment

Figure 12 shows logging into a new docker account and then pulling the image from the public repo.

Running Docker container on different environments



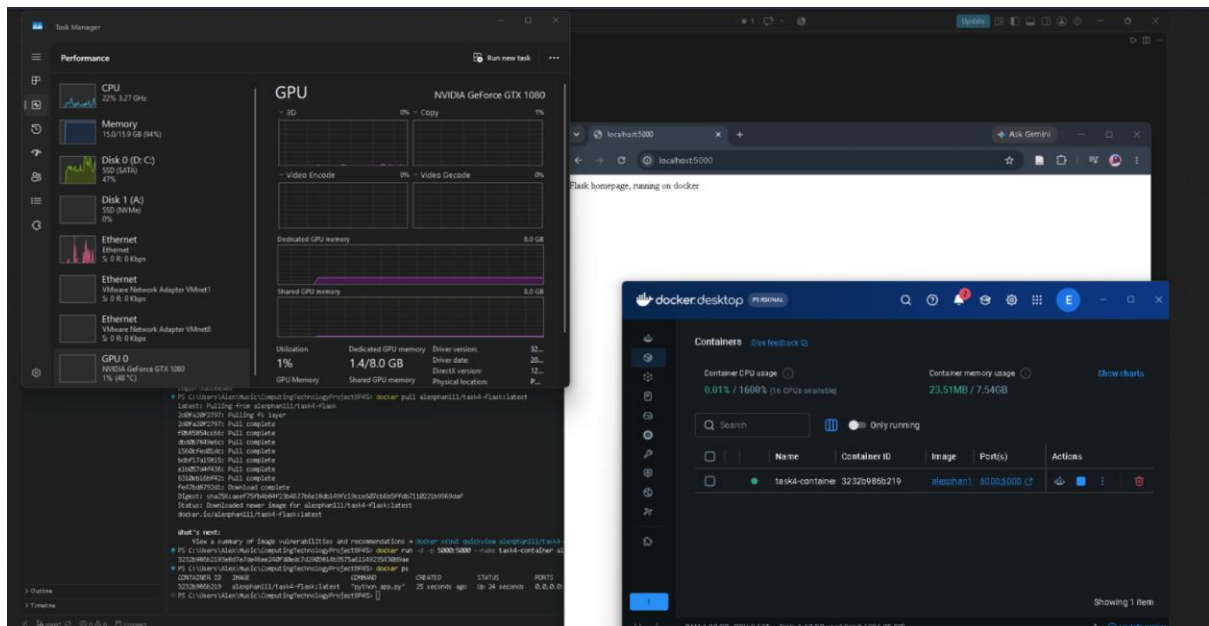
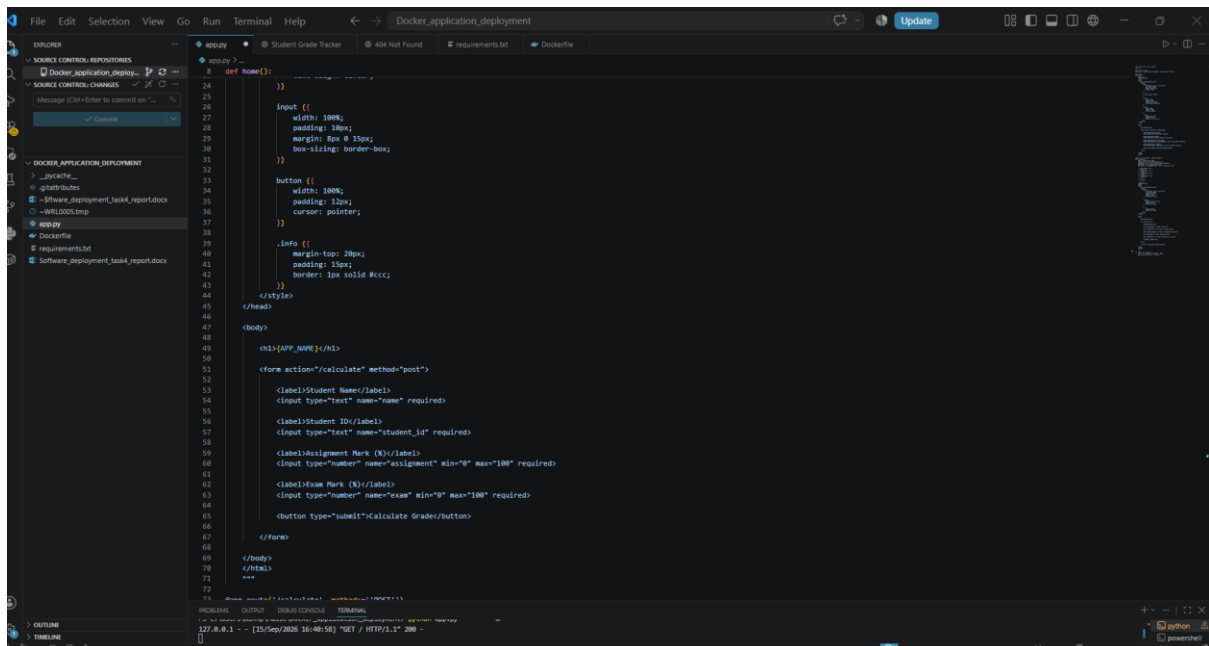


Figure 13, Different environments

Figure 13 highlights the containers being able to run on different environments after demonstrating that the image of the flask web app can be pulled from the public docker repository online and used to spin up a container. Furthermore, this is shown through the different computing specifications shown in task manager which is evidence of multi environment.

Distinction Level Task:

Grade calculation front page.



Student Name

Student ID

Assignment Mark (%)

Exam Mark (%)

Calculate Grade

Figure 14, Front page

For the beginning of the distinction task, I have decided to create a simple student grade tracker app, figure 14 shows the use of a html form tag to create the grade calculation form which will send a post request to the backend. Additionally, I have configured environment variables for the title of the page so that Docker can configure them.

Grade calculated page.

Student Grade Tracker

Grade Result

Student: alex

Student ID: 10231231

Assignment: 30.0%

Exam: 70.0%

Overall: 54.0%

Grade: P

[Calculate another grade](#)

```
PS C:\Users\minhp\Music\docker_application_deployment> python app.py
127.0.0.1 - - [15/Sep/2026 16:46:25] "POST /calculate HTTP/1.1" 200 -
127.0.0.1 - - [15/Sep/2026 16:46:31] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [15/Sep/2026 16:46:41] "POST /calculate HTTP/1.1" 200 -
```

```

72 @app.route('/calculate', methods=['POST'])
73 def calculate():
74     name = request.form['name']
75     student_id = request.form['student_id']
76     assignment_mark = float(request.form['assignment'])
77     exam_mark = float(request.form['exam'])
78     total_score = (assignment_mark * 0.4) + (exam_mark * 0.6)
79
80     if total_score >= 80:
81         grade='HD'
82     elif total_score >= 70:
83         grade='D'
84     elif total_score >= 60:
85         grade='C'
86     elif total_score >= 50:
87         grade='P'
88     else:
89         grade='F'
90
91     return f"""
92     <doctype html>
93     <html>
94     <head>
95     <title>{APP_NAME}</title>
96     <style>
97     <body>{{
98         font-family: Arial, sans-serif;
99         max-width: 600px;
100        margin: 50px auto;
101        padding: 20px;
102        text-align: center;
103    }}
104
105    .result {{
106        border: 1px solid #ccc;
107        padding: 25px;
108        margin-top: 20px;
109    }}
110
111    a {{
112        display: block;
113        margin-top: 20px;
114    }}
115    </style>
116    </head>
117
118

```

Figure 15, Grade calculated page

Figure 15 shows the route code where the flask server listens for the POST request on /calculate page, after receiving POST request it will then calculate the grades entered in the forms using the names of the tags within the form html in the front page.

Update Docker file.

```

1 FROM python:3.12-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6
7 RUN pip install -r requirements.txt
8
9 COPY app.py .
10
11 ENV PORT=5000
12 ENV APP_NAME='Grade Calculator'
13
14 EXPOSE 5000
15
16 CMD ["python", "app.py"]

```

Figure 16, Update Docker file

I then updated the docker file so that it would set the environment variables instead of me having to change the program in the code every time, I can now change it in the Docker file.

Build Image

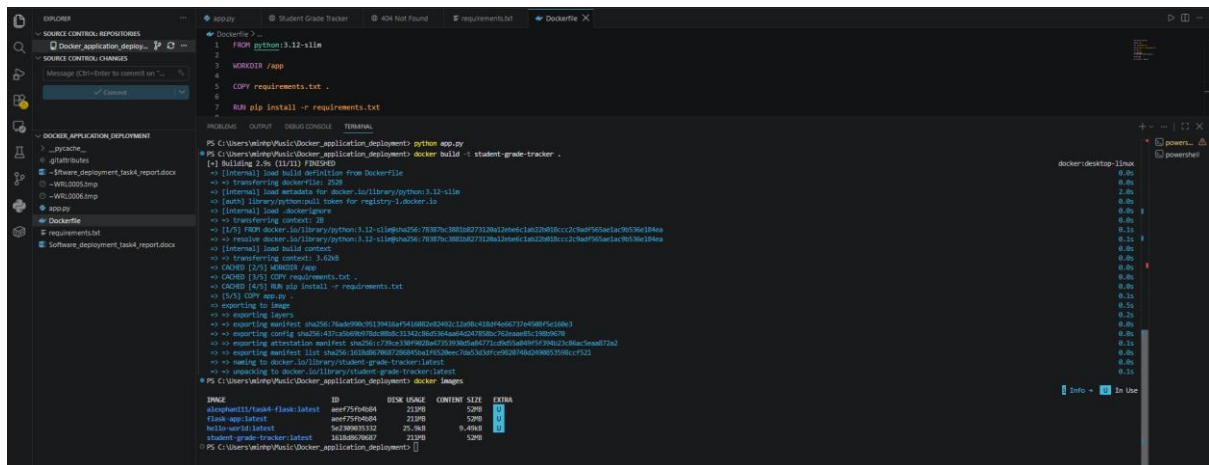


Figure 17, Build image.

After developing the main program, I then used docker built to create the docker image which I will then use to recreate my application within a docker container.

Docker Container

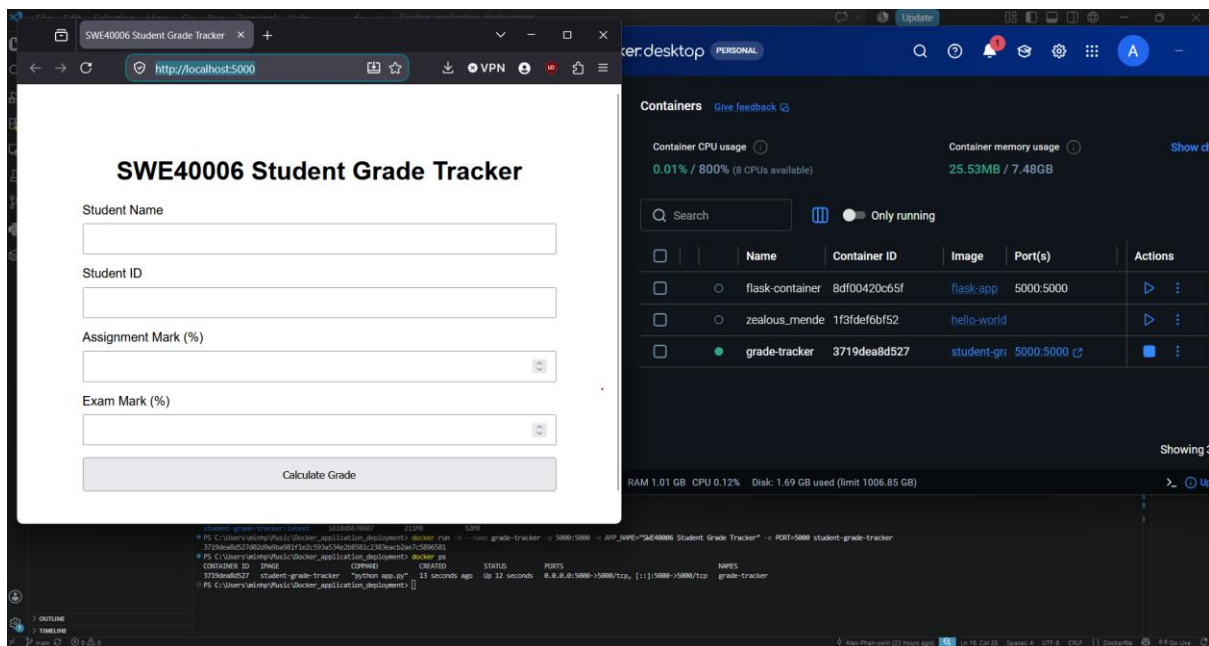


Figure 18, Container

Figure 18 shows the application running on the registered docker container named grade-tracker which was created based on the applications image shown in figure 17.

Docker Network

```
PS C:\Users\minhp\Music\docker_application_deployment> docker network create grade-network
b802ae58d37694487d1eeaf9b37b2a53cf98782998154ffca422cb4789c
PS C:\Users\minhp\Music\docker_application_deployment> docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
b8792880773b        grade-network       bridge              local
b802ae58d376        grade-network       bridge              local
e6e824c8f6e6        host                host                local
da02a9666599        none               null                local
PS C:\Users\minhp\Music\docker_application_deployment> docker run -d --name grade-tracker --network grade-network -p 5000:5000 student-grade-tracker
docker: Error response from daemon: Conflict. The container name "/grade-tracker" is already in use by container "3719de8d527d02d9a9a81f1e2c593a534e2b8381c2383eacb2ae7c5896581". You have to remove (or rename) that container to be able to reuse that name.
Run 'docker run --help' for more information
PS C:\Users\minhp\Music\docker_application_deployment> docker run -d --name grade-tracker --network grade-network -p 5000:5000 student-grade-tracker
docker: Error response from daemon: Conflict. The container name "/grade-tracker" is already in use by container "3719de8d527d02d9a9a81f1e2c593a534e2b8381c2383eacb2ae7c5896581". You have to remove (or rename) that container to be able to reuse that name.
Run 'docker run --help' for more information
PS C:\Users\minhp\Music\docker_application_deployment> docker run -d --name grade-tracker --network grade-network -p 5000:5000 student-grade-tracker
7b6dad40ca4d75c7fca2928ff51ad14d81562b28dad23c09efcf83215
PS C:\Users\minhp\Music\docker_application_deployment>
```

Figure 19, Network

I have also deleted the previous docker container and recreated a new container using a new resolute docker network for the student grade app. The purpose of the docker network is so that it provides communication between containers but also external networks.

Push to repo

```
PS C:\Users\minhp\Music\docker_application_deployment> docker login
Authenticating with existing credentials... [Username: alexphan111]

Info -> To Login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
PS C:\Users\minhp\Music\docker_application_deployment> docker tag student-grade-tracker alexphan111/student-grade-tracker:latest
PS C:\Users\minhp\Music\docker_application_deployment> docker push alexphan111/student-grade-tracker:latest
The push refers to repository [docker.io/alexphan111/student-grade-tracker]
f0b85854cc66: Mounted from alexphan111/task4-flask
2e55b1e87cd5: Pushed
1560cfed01dc: Mounted from alexphan111/task4-flask
6310eb16bf42: Mounted from alexphan111/task4-flask
dbd0b7849e6c: Mounted from alexphan111/task4-flask
56235e424563: Mounted from alexphan111/task4-flask
146744a9ce27: Pushed
bdbf17a15815: Mounted from alexphan111/task4-flask
e1b057d4f436: Mounted from alexphan111/task4-flask
alexphan111/student-grade-tracker:latest: digest sha256:1561494870827185845e3d56f320ee74d5747d5e0892748d4d0083f88e0f501 -> sha256:1561494870827185845e3d56f320ee74d5747d5e0892748d4d0083f88e0f501
```

Figure 20, Pushing image.

Figure 20 shows the code that pushes the student grading application to docker desktop, this is so that when I run an ec2 I can pull the container and run that on the ec2 which can make the application accessible through a public Ip address.

Security Group

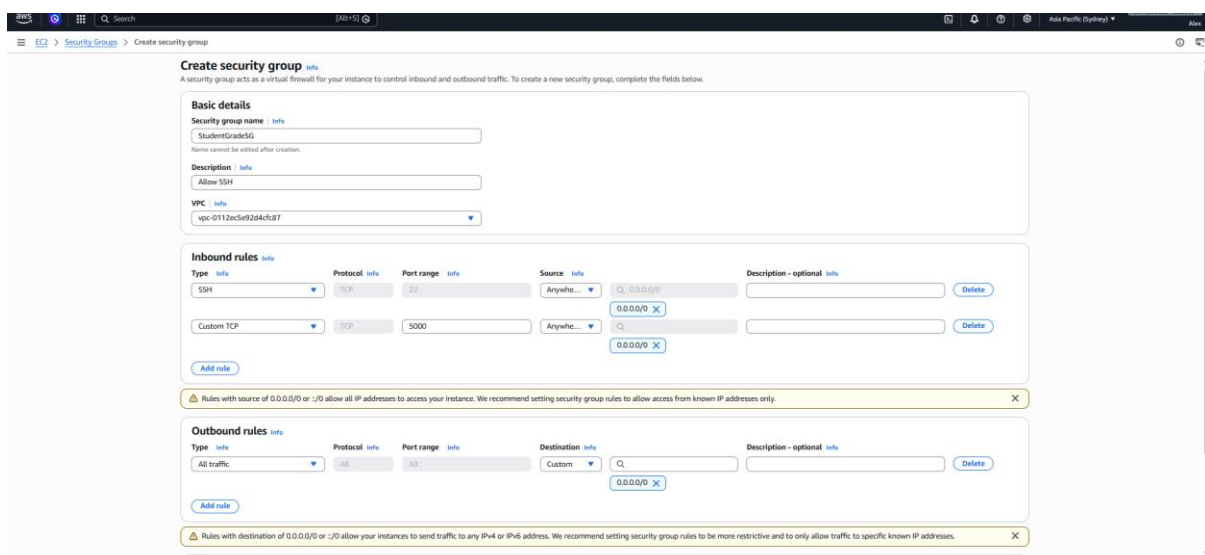


Figure 21, Security group

Figure 21 shows the security group for the ec2 instance, it lets ssh for me to pull the container from docker hub but also the port 5000 tcp since the flask application will be listening for port 5000.

EC2

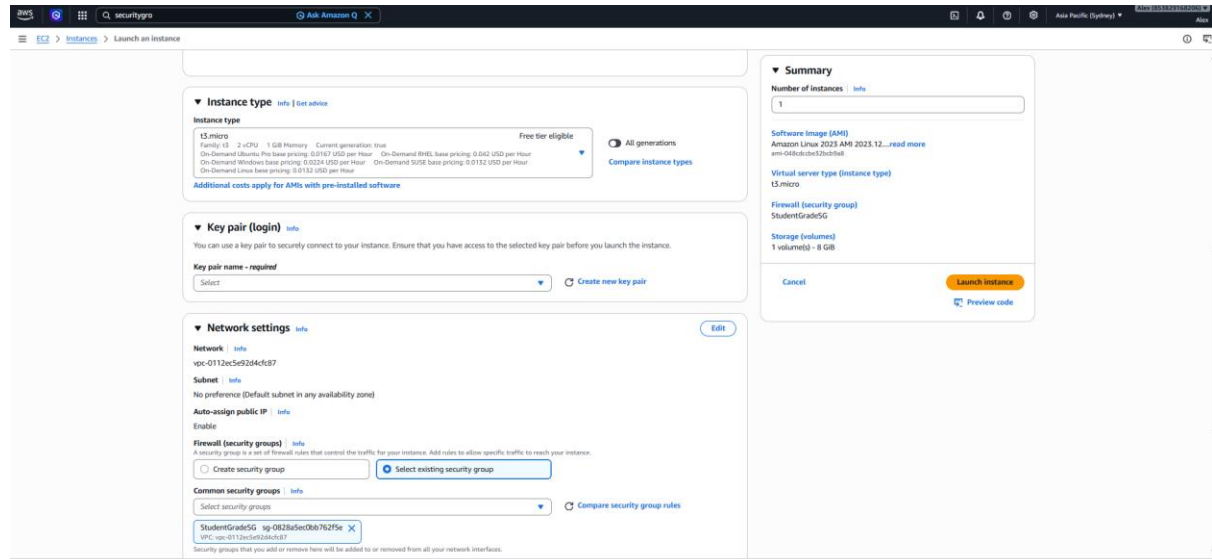


Figure 22, EC2

I now launch an ec2 instance using the created security group in figure 21 with the default settings for amazon Linux.

Install Docker

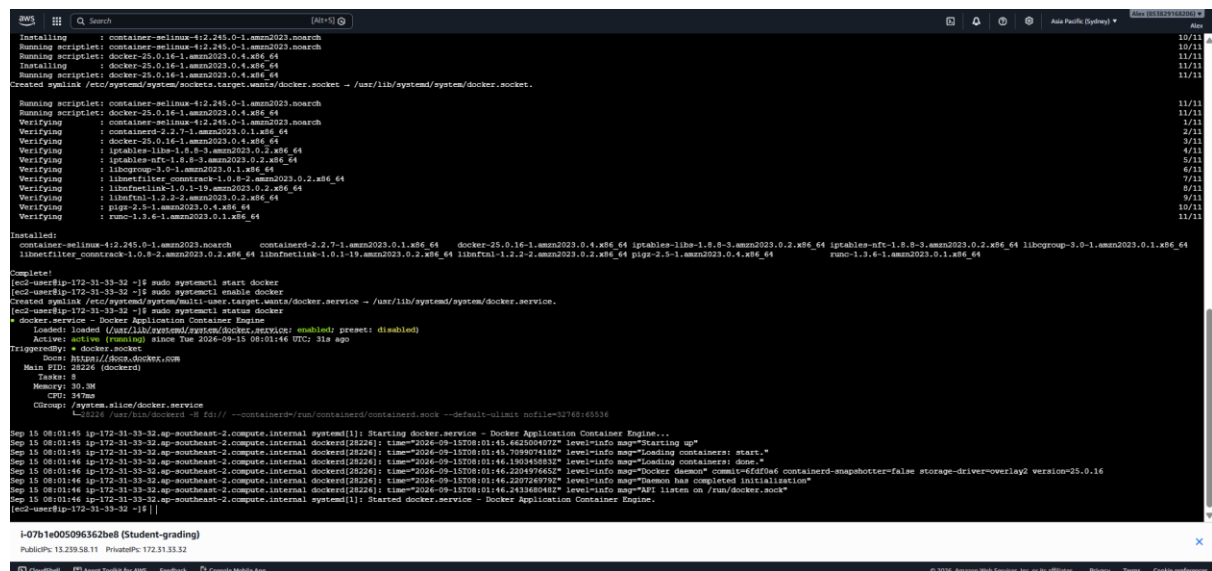


Figure 23, Docker installation

Install docker on the VM so that we can pull the image and run the container.

Docker Container on ec2

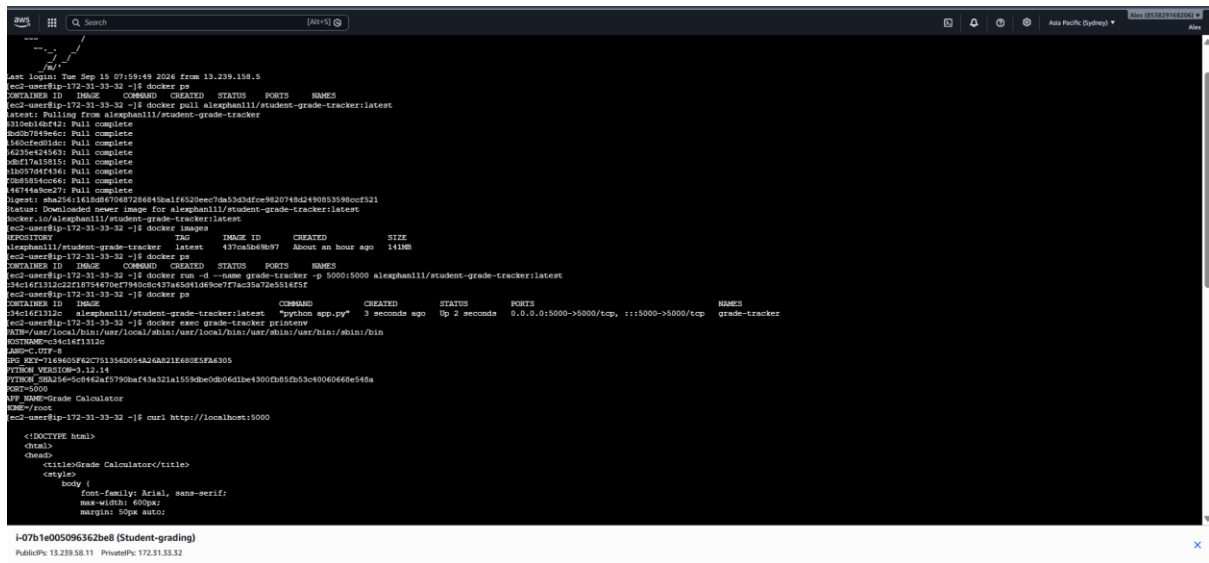


Figure 24, Pulling image.

Figure 24 shows me pulling the image from the public docker hub repo and then creating a container on the VM using the student grading image.

Publicly accessible application

← → ↻ ⚠ Not secure 13.239.58.11-5000 ☆ 📄 🗑 🖨 🔽 🌐

Grade Calculator

Student Name

Student ID

Assignment Mark (%)

Exam Mark (%)

Calculate Grade

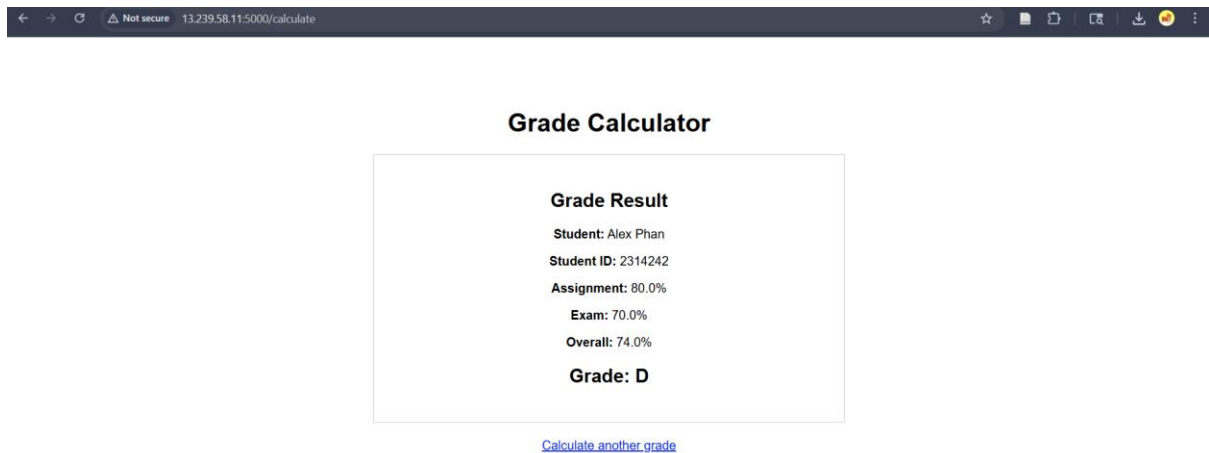


Figure 25, Public site

Figure 25 shows the student grade app being publicly accessible on a public site, there was an issue with this part as the flask application only received http request and not https. Additionally, the link had to specify which port to connect to the ec2 on, I had accessed the ec2 through the public Ip however this could not route me to the app since in the app I had specified to only listen for port 5000. The app is accessible at `http://13.239.58.11:5000`.

External Code & Script Repository

GitHub repository: https://github.com/Alex-Phan-swin/Docker_application_deployment